Experiment 6

Student Name: Akshit Gautam **UID:** 23BAI71449
Branch: BE CSE AIML **Section/Group:** 23AIT-KRG G1
Semester: 6th **Date of Performance:** 7 March 2026
Subject Name: Full Stack **Subject Code:** 23CSH-382

1. **Title:**

   Spring Boot REST API – HealthHub

2. **Aim:**
   To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

3. **Objective:**
   After completing this experiment, students will be able to:
   1. Understand Spring Boot fundamentals
      - Learn the role of Spring Boot in backend development.
      - Configure and run a basic Spring Boot application.
   2. Apply IoC and Dependency Injection
      - Understand how Spring manages object creation and dependencies.
      - Use annotations like @Autowired, @Service, and @Repository.
   3. Implement Layered Architecture
      - Structure the application using Controller, Service, and Repository layers.
      - Understand separation of concerns in backend design.
   4. Develop RESTful APIs

    - Create CRUD operations using Spring Boot REST controllers.
    - Design APIs using proper HTTP methods (GET, POST, PUT, DELETE).
5. Use DTOs for API communication
    - Implement DTO mapping to separate internal models from API responses.
6. Implement Validation
    - Apply validation using Jakarta Validation annotations (@NotNull, @Email, etc.).
7. Handle Exceptions Globally
    - Implement global exception handling using @ControllerAdvice.
8. Build the HealthHub Backend Module
    - Develop backend services for patient/health record management.

4. Implementation/Code:

- Patient.java:

```java
package com.healthhub.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor

public class Patient {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private int age;

    private String disease;
}
```

- PatientDTO.java:

```java
package com.healthhub.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data  24 usages
@NoArgsConstructor
@AllArgsConstructor

public class PatientDTO {
    private Long id;

    @NotBlank(message = "Name cannot be empty")
    private String name;

    @Email(message = "Invalid Email")
    private String email;

    @Min(value = 1, message = "Age must be valid")
    private int age;

    @NotBlank(message = "Disease field required")
    private String disease;
}
```

- PatientRepo.java:

```java
package com.healthhub.repository;

import com.healthhub.entity.Patient;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PatientRepo extends JpaRepository<Patient, Long> {

}
```

- PatientController.java:

```java
package com.healthhub.controller;

import com.healthhub.dto.PatientDTO;
import com.healthhub.service.PatientService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/patients")
public class PatientController {

    @Autowired
    private PatientService patientService;

    @PostMapping
    public PatientDTO createPatient(@Valid @RequestBody PatientDTO patientDTO) {
        return patientService.createPatient(patientDTO);
    }

    @GetMapping
    public List<PatientDTO> getAllPatients() {
        return patientService.getAllPatients();
    }

    @GetMapping("/{id}")
    public PatientDTO getPatientById(@PathVariable Long id) {
        return patientService.getPatientById(id);
    }

    @PutMapping("/{id}")
    public PatientDTO updatePatient(@PathVariable Long id,
                                    @Valid @RequestBody PatientDTO patientDTO) {
        return patientService.updatePatient(id, patientDTO);
    }

    @DeleteMapping("/{id}")
    public String deletePatient(@PathVariable Long id) {
        patientService.deletePatient(id);
        return "Patient deleted successfully";
    }
}
```

- PatientService.java:

```java
package com.healthhub.service;

import com.healthhub.dto.PatientDTO;
import java.util.List;

public interface PatientService {  3 usages  1 implementation
    PatientDTO createPatient(PatientDTO patientDTO);  1 usage  1 impleme

    List<PatientDTO> getAllPatients();  1 usage  1 implementation

    PatientDTO getPatientById(Long id);  1 usage  1 implementation

    PatientDTO updatePatient(Long id, PatientDTO patientDTO);  1 usage

    void deletePatient(Long id);  1 usage  1 implementation
}
```

- PateintServiceImpl.java:

```java
package com.healthhub.service;

import com.healthhub.dto.PatientDTO;
import com.healthhub.entity.Patient;
import com.healthhub.repository.PatientRepo;
import com.healthhub.exception.ResourceNotFoundException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class PatientServiceImpl implements PatientService {
    @Autowired
    private PatientRepo patientRepository;
    private PatientDTO mapToDTO(Patient patient) {  4 usages
        return new PatientDTO(
                patient.getId(),
                patient.getName(),
                patient.getEmail(),
                patient.getAge(),
                patient.getDisease()
        );
    }
```

```java
    private Patient mapToEntity(PatientDTO dto) {  1 usage
        return new Patient(
                dto.getId(),
                dto.getName(),
                dto.getEmail(),
                dto.getAge(),
                dto.getDisease()
        );
    }
    @Override  1 usage
    public PatientDTO createPatient(PatientDTO patientDTO) {
        Patient patient = mapToEntity(patientDTO);
        Patient saved = patientRepository.save(patient);
        return mapToDTO(saved);
    }
    @Override  1 usage
    public List<PatientDTO> getAllPatients() {
        return patientRepository.findAll()  List <Patient >
                .stream()  Stream <Patient >
                .map(this::mapToDTO)  Stream <PatientDTO >
                .collect(Collectors.toList());
    }
    @Override  1 usage
    public PatientDTO getPatientById(Long id) {
        Patient patient = patientRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Patient not found"));
        return mapToDTO(patient);
    }
    @Override  1 usage
    public PatientDTO updatePatient(Long id, PatientDTO dto) {
        Patient patient = patientRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Patient not found"));
        patient.setName(dto.getName());
        patient.setEmail(dto.getEmail());
        patient.setAge(dto.getAge());
        patient.setDisease(dto.getDisease());
        Patient updated = patientRepository.save(patient);
        return mapToDTO(updated);
    }
    @Override  1 usage
    public void deletePatient(Long id) {
        Patient patient = patientRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Patient not found"));
        patientRepository.delete(patient);
    }
}
```

- GlobalExceptionHandler.java:

```java
package com.healthhub.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public String handleResourceNotFound(ResourceNotFoundException ex) {
        return ex.getMessage();
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public String handleGeneralException(Exception ex) {
        return "Something went wrong: " + ex.getMessage();
    }
}
```

- ResourceNotFoundException.java:

```java
package com.healthhub.exception;

public class ResourceNotFoundException extends RuntimeException {  6 us

    public ResourceNotFoundException(String message) {  3 usages
        super(message);
    }
}
```

- HealthHubApplication,http:

```
POST http://localhost:8080/api/patients
Content-Type: application/json

{
  "name": "John",
  "email": "john@email.com",
  "age": 30,
  "disease": "Flu"
}

<> 2026-03-15T175013.200.json

###

GET http://localhost:8080/api/patients

<> 2026-03-15T175013-1.200.json

###

GET http://localhost:8080/api/patients/1

<> 2026-03-15T175013-2.200.json
```

5. Output

```
{
  "id": 1,
  "name": "John",
  "email": "john@email.com",
  "age": 30,
  "disease": "Flu"
}
```

Fig1. Post Request

```
[
  {
    "id": 1,
    "name": "John",
    "email": "john@email.com",
    "age": 30,
    "disease": "Flu"
  }
]
```

```
{
  "id": 1,
  "name": "John",
  "email": "john@email.com",
  "age": 30,
  "disease": "Flu"
}
```

Fig 2. Get requests (2.1: GET Patients; 2.2: GET patients/1)

6. Learning Outcome

- We learnt about Spring.
- We learnt about Spring Architecture.
- We learnt about GET, POST and PUT requests.
- We learnt about Exception handling in Spring.
- We learnt about various Spring dependencies.